



Guía de Estudio: Interfaces Gráficas en Java

Desarrollo con JavaFX, IntelliJ IDEA, FXML y Arquitectura MVC (Indicadores IL 2.1 – IL 2.8)

Objetivo Metodológico: La idea es que no solamente sepas qué código escribir, sino que entiendas por qué existe cada parte y cómo se relacionan entre sí, porque eso es precisamente lo que evalúan los indicadores IL 2.1–2.8.

TECNOLOGÍAS FUNDAMENTALES

Tecnología	Para qué sirve
IntelliJ IDEA	Entorno Integrado de Desarrollo (IDE) donde se desarrolla y ejecuta el proyecto.
Java	Lenguaje de programación orientado a objetos base.
JavaFX	Framework moderno para la creación de interfaces gráficas de usuario (GUI).
FXML	Lenguaje de marcado basado en XML para definir la interfaz visualmente.
Scene Builder	Herramienta WYSIWYG para diseñar componentes FXML de manera gráfica.
Maven	Gestor de dependencias y automatización de la construcción del proyecto.
MVC	Patrón de arquitectura que separa Modelo, Vista y Controlador.
Serializable	Interfaz que permite la conversión de objetos en flujo de bytes para guardar en archivos.
DAO	Patrón Data Access Object que aísla la persistencia de la lógica de negocio.

1. POO APLICADA A INTERFACES GRÁFICAS

Una aplicación gráfica sigue siendo un programa orientado a objetos. La interfaz gráfica **no reemplaza** a la POO, la **utiliza** para interactuar con el usuario.

```
public class Cliente {
    private String nombre;
    private String correo;

    public Cliente(String nombre, String correo) {
        this.nombre = nombre;
        this.correo = correo;
    }

    public String getNombre() { return nombre; }
    public String getCorreo() { return correo; }
}
```

Conceptos Aplicados

Clase: Modelo del dominio (Cliente).

Encapsulamiento: Atributos `private` con getters.

Instancia: Objetos procesados en memoria.

Relación con la GUI

Los componentes de la interfaz (como un `TableView`) consumen directamente listas de estos objetos para presentar la información al usuario.

2 - 4. CONFIGURACIÓN DEL PROYECTO (IL 2.1)

5 - 6. JAVA FX: ESTRUCTURA STAGE, SCENE Y NODE IL 2.1

La jerarquía visual de JavaFX se puede analogar con una obra de teatro o un edificio:

Stage (Escenario)

Representa la ventana principal del sistema operativo (marco exterior con botones de cerrar/maximizar).

Scene (Escena)

Es el contenedor del contenido visible dentro de la ventana. Se puede intercambiar para cambiar de pantalla.

Estructura jerárquica: Stage → Scene → Node Raíz (VBox/GridPane) → Nodes Hijos (Button, Label, TextField).

Stage (Ventana)



Scene (Escena)



VBox (Contenedor)



Nodes (Button, Label)

7. CICLO DE VIDA DE UNA APLICACIÓN JAVA FX

Toda aplicación JavaFX principal hereda de la clase abstracta `javafx.application.Application`.

```
public class Main extends Application {
    @Override
    public void start(Stage stage) throws Exception {
        // Carga de la escena inicial y despliegue de la ventana
        FXMLLoader loader = new FXMLLoader(getClass().getResource("/com/ejemplo/principal.fxml"));
        Parent root = loader.load();
        Scene scene = new Scene(root);
        stage.setScene(scene);
        stage.setTitle("Sistema de Gestión");
        stage.show();
    }

    public static void main(String[] args) {
        launch(args); // Inicia el runtime de JavaFX
    }
}
```

Secuencia de ejecución interna:

main()



launch()



JavaFX Init



start(Stage)



Event Loop

8. CARGA DE INTERFACES FXML

Para desacoplar el diseño visual del código Java, se utiliza la clase `FXMLLoader` para instanciar la vista declarativa:

```
FXMLLoader loader = new FXMLLoader(getClass().getResource("/com/ejemplo/principal.fxml"));
Parent root = loader.load(); // Construye el árbol de Nodes en memoria
Scene scene = new Scene(root);
stage.setScene(scene);
stage.show();
```

Resumen de Clases Base de JavaFX

- Button, Label, TextField, PasswordField: Controles básicos de entrada/salida.
- ComboBox, CheckBox, RadioButton, DatePicker: Controles de selección.
- TableView, TableColumn: Representación masiva de datos en formato tabular.

9 - 12. MARCADO DECLARATIVO FXML Y SCENE BUILDER IL 2.2

Indicador IL 2.2: Diseña interfaces mediante herramientas visuales declarativas vinculando componentes con el controlador.

FXML: Lenguaje basado en XML donde se estructuran los componentes visuales de manera declarativa sin lógica de programación.

```
<!-- principal.fxml -->
<VBox spacing="10" alignment="CENTER" xmlns:fx="http://javafx.com/fxml"
    fx:controller="com.ejemplo.controller.PrincipalController">
    <Label text="Registro de Cliente"/>
    <TextField fx:id="txtNombre" promptText="Ingrese Nombre"/>
    <Button text="Guardar" onAction="#guardar"/>
</VBox>
```

IntelliJ (Proyecto)



Archivo .fxml



Scene Builder (Diseño)



Java Controller

13 - 16. EL CONTROLLER Y LA ANOTACIÓN @FXML

El enlace entre la vista declarativa FXML y el código ejecutable Java se realiza mediante el atributo `fx:controller` y los vinculadores `fx:id` y `onAction`.

```
package com.ejemplo.controller;

import javafx.fxml.FXML;
import javafx.scene.control.TextField;

public class PrincipalController {

    // Vincula el componente con fx:id="txtNombre"
    @FXML
    private TextField txtNombre;

    // Vincula el método con el evento onAction="#guardar"
    @FXML
    private void guardar() {
        String nombre = txtNombre.getText();
        System.out.println("Cliente registrado: " + nombre);
    }
}
```

fx:id

Identificador único asignado al componente visual en FXML para poder manipularlo programáticamente desde Java.

@FXML

Anotación de reflexividad que otorga acceso a JavaFX a atributos y métodos privados dentro del Controller.

Acción del Usuario (Clic)



onAction="#guardar"



@FXML private void guardar()

Indicador IL 2.3: Captura, valida e interactúa con datos de componentes y manejo de eventos.

Sintaxis Básica de Manipulación de Componentes

```
// Captura y asignación de texto
String texto = txtNombre.getText();
lblMensaje.setText("Bienvenido " + texto);

// Captura de controles de selección
boolean activo = chkActivo.isSelected();
String ciudad = cmbCiudad.getValue();
LocalDate fecha = datePicker.getValue();
```

Validación Formulario Completo y Excepciones

Las interfaces deben validar el tipo de dato y la presencia de información requerida antes de instanciar objetos del dominio.

```
@FXML
private void guardar() {
    // 1. Validar campos obligatorios vacíos
    if (txtNombre.getText().isBlank()) {
        mostrarError("El nombre es obligatorio.");
        return;
    }

    // 2. Validar formato numérico mediante try-catch
    try {
        int edad = Integer.parseInt(txtEdad.getText());
        if (edad < 18) {
            mostrarError("El cliente debe ser mayor de edad.");
            return;
        }
    } catch (NumberFormatException e) {
        mostrarError("La edad debe ser un número entero válido.");
        return;
    }

    System.out.println("Formulario procesado con éxito.");
}
```

Uso de Diálogos con la Clase Alert

JavaFX provee la clase `Alert` para desplegar ventanas emergentes modales para retroalimentar al usuario:

```
private void mostrarError(String mensaje) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Error de Validación");
    alert.setHeaderText(null);
    alert.setContentText(mensaje);
    alert.showAndWait(); // Bloquea la interacción hasta aceptar
}
```

Alert.AlertType.INFORMATION

Avisos de éxito o confirmación de proceso completado.

Alert.AlertType.WARNING

Advertencias sin bloqueo de flujo crítico.

Alert.AlertType.CONFIRMATION

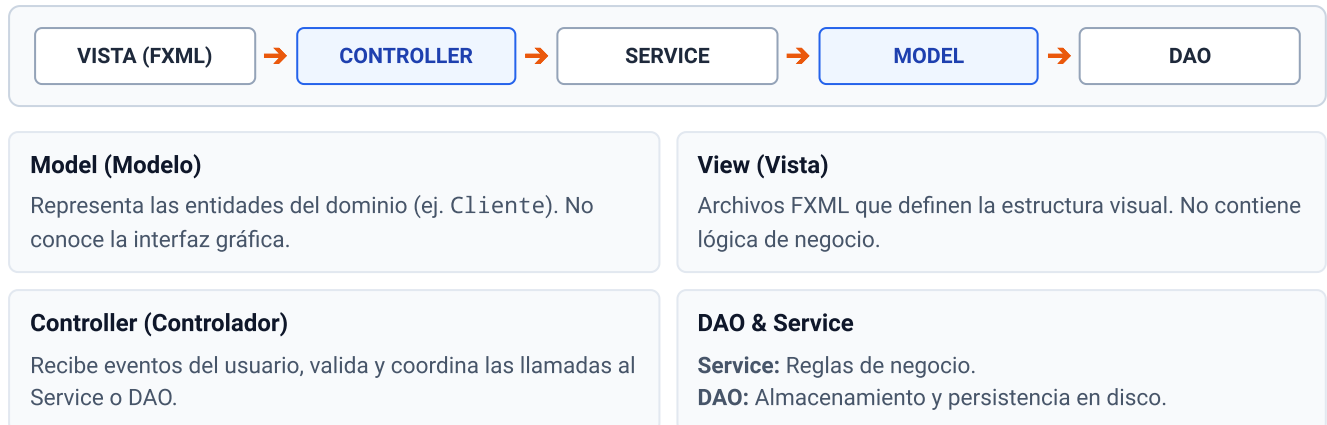
Requiere respuesta Aceptar / Cancelar del usuario.

Alert.AlertType.ERROR

Informa fallos fatales o errores de validación.

20 - 27. ARQUITECTURA PATRÓN MVC + LAYERING IL 2.4

Indicador IL 2.4: Aplica el patrón Modelo-Vista-Controlador separando responsabilidades.



Estructura Organizativa del Proyecto en IntelliJ IDEA

```
src/main/ ├── java/com/hotel/ | ├── Main.java (Clase Application principal) | ├──
model/Cliente.java (Clases Entidad) | ├── controller/ (Controladores JavaFX) | | ├──
PrincipalController.java | | ├── ClienteController.java | ├── service/ClienteService.java | ├──
dao/ClienteDAO.java (Gestión de Archivos) | ├── resources/com/hotel/ (Vistas FXML e imágenes) | ├──
principal.fxml | ├── cliente.fxml
```

28 - 32. DESPLIEGUE DE COLECCIONES EN TABLEVIEW IL 2.5

Indicador IL 2.5: Muestra colecciones de datos en componentes tabulares utilizando colecciones observables.

```
@FXML private TableView<Cliente> tablaClientes;
@FXML private TableColumn<Cliente, String> colNombre;
@FXML private TableColumn<Cliente, String> colCorreo;

private ObservableList<Cliente> listaClientes = FXCollections.observableArrayList();

@FXML
private void initialize() {
    // Configura la extracción de atributos desde el modelo para cada columna
    colNombre.setCellValueFactory(data -> new SimpleStringProperty(data.getValue().getNombre
()));
    colCorreo.setCellValueFactory(data -> new SimpleStringProperty(data.getValue().getCorreo
()));

    // Enlaza la lista observable con la tabla
    tablaClientes.setItems(listaClientes);
}
```

33 - 35. NAVEGACIÓN ENTRE VISTAS Y TRANSFERENCIA IL 2.6

Indicador IL 2.6: Implementa mecanismos de navegación entre múltiples vistas y pasa datos entre controladores.

Cambio de Pantalla y Paso de Parámetros

```
// 1. Cargar el FXML de la vista destino
FXMLLoader loader = new FXMLLoader(getClass().getResource("/com/hotel/detalle.fxml"));
Parent root = loader.load();

// 2. Obtener la instancia del Controller destino
DetalleController controllerDestino = loader.getController();

// 3. Transferir el objeto seleccionado
Cliente seleccionado = tablaClientes.getSelectionModel().getSelectedItem();
controllerDestino.setCliente(seleccionado);

// 4. Cambiar la escena en el Stage actual
Stage stage = (Stage) ((Node) event.getSource()).getScene().getWindow();
stage.setScene(new Scene(root));
stage.show();
```

Vista A (Selección)



Controller A



loader.getController()



Controller B (setCliente)

36 - 43. PERSISTENCIA DE DATOS MEDIANTE SERIALIZACIÓN IL 2.7

Indicador IL 2.7: Implementa almacenamiento persistente en archivos de objeto (Serialización y DAO).

Para guardar objetos en disco, la clase modelo debe implementar la interfaz marcadora Serializable:

```
public class Cliente implements Serializable {
    private static final long serialVersionUID = 1L; // Garantiza compatibilidad
    private String nombre;
    private String correo;
    // Constructor y getters...
}
```

Implementación del Patrón DAO (Data Access Object)

```
public class ClienteDAO {
    private final String rutaArchivo = "clientes.dat";

    public void guardar(List<Cliente> clientes) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(rutaArchivo))) {
            out.writeObject(clientes); // Serializa la lista completa
        }
    }

    @SuppressWarnings("unchecked")
    public List<Cliente> cargar() throws IOException, ClassNotFoundException {
        File file = new File(rutaArchivo);
        if (!file.exists()) return new ArrayList<>(); // Si no existe, retorna lista vacía

        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(file))) {
            return (List<Cliente>) in.readObject(); // Deserialización
        }
    }
}
```

44 - 47. INTEGRACIÓN COMPLETA DEL PROYECTO IL 2.8

Indicador IL 2.8: Desarrolla un componente de software completo aplicando el patrón MVC con persistencia en archivo.

Flujo Completo del Sistema de Gestión

Al arrancar, el controlador llama al DAO para deserializar `clientes.dat` e hidratar el `ObservableList` del `TableView`. Al presionar "Guardar", se valida la entrada, se crea el objeto, se actualiza la vista y se guarda en archivo.

Main (launch)



DAO.cargar()



ObservableList



TableView UI

48. MATRIZ DE COBERTURA POR INDICADOR DE LOGRO

IL 2.1 – Configuración y Entorno

Conocer IntelliJ, JavaFX, Maven, ciclo de vida (launch/start) y jerarquía Stage/Scene/Node.

IL 2.2 – Marcado FXML

Dominar Scene Builder, sintaxis FXML, `fx:controller`, `fx:id`, `@FXML` y eventos `onAction`.

IL 2.3 – Componentes y Validación

Manipular Label, TextField, ComboBox, DatePicker, Alert y bloques try-catch de validación.

IL 2.4 – Patrón MVC

Separar claramente entidades (Model), pantallas (View FXML) y lógica de eventos (Controller).

IL 2.5 – TableView y Colecciones

Asociar `ObservableList` a `TableView` y configurar `CellValueFactory` para mostrar objetos.

IL 2.6 – Navegación

Uso de `FXMLLoader` para cambiar de pantalla y transferir instancias entre controladores.

IL 2.7 – Persistencia DAO

Uso de `Serializable`, `ObjectOutputStream/ObjectInputStream` y aislamiento en clase DAO.

IL 2.8 – Sistema Integrado

Ensamblaje final de una aplicación CRUD completa funcional y libre de errores.

49 - 50. DIAGNÓSTICO DE ERRORES COMUNES

Error / Excepción	Causa Probable y Solución
"FXML no encontrado"	Ruta incorrecta en <code>getResource()</code> . Debe estar en <code>src/main/resources</code> respetando el paquete.
@FXML NullPointerException	El <code>fx:id</code> especificado en FXML no coincide exactamente con el nombre de la variable en el Controller.
El botón no ejecuta acción	Falta definir <code>onAction="#metodo"</code> en FXML o el método no tiene la anotación @FXML.
Controller no encontrado	El atributo <code>fx:controller="..."</code> en FXML no coincide con el nombre de paquete/clase Java.
NumberFormatException	Falló la conversión de texto a número (ej. <code>Integer.parseInt()</code>) por entrada inválida. Usar try-catch.
Tabla vacía al abrir	Faltó invocar <code>tabla.setItems(lista)</code> o los <code>cellValueFactory</code> están mal configurados.

51. RUTA DE ESTUDIO SUGERIDA EN 7 ETAPAS



52. CHECKLIST DE AUTOEVALUACIÓN FINAL

- ¿Sé explicar la diferencia entre Stage, Scene y Node?
- ¿Puedo conectar un campo de texto de Scene Builder con una variable Java mediante `fx:id` y @FXML?
- ¿Sé cómo usar FXMLLoader para abrir una nueva ventana y enviarle un objeto seleccionado?
- ¿Por qué el Controller no debe guardar directamente los datos en archivos sino usar una clase DAO?
- ¿Qué función cumple ObservableList en relación con la actualización visual del TableView?



Idea Central para la Evaluación

La interfaz no es el programa completo, es la puerta de entrada. El objetivo de los indicadores IL 2.1–2.8 es construir aplicaciones donde la interfaz, los objetos, la lógica y los datos estén correctamente separados y conectados.